



Data Glacier

Data Science Intern at Data Glacier

Week 5: Cloud and API deployment

Name: Mehedi Hasan Shakil

Batch Code: LISUM44

Date: 03 May 2025

Submitted to: Data Glacier

Table of Contents:

1. Introduction.....	2
2. Data Information.....	2
2.1. Attribute Information.....	3
3. Build Machine Learning Model.....	4
4. Turning Model into Flask Framework.....	6
4.1. App.py.....	7
4.2. Index.html.....	9
4.3. Style.css.....	10
4.4. Running Procedure.....	11
5. Model Development using Heroku	13
5.1. Steps for Model Development using Heroku	13

1. Introduction

In this project, we are going to deploy a machine learning model (Random Forest) using the Flask Framework. As a demonstration, our model helps to predict fake and genuine accounts on Instagram.

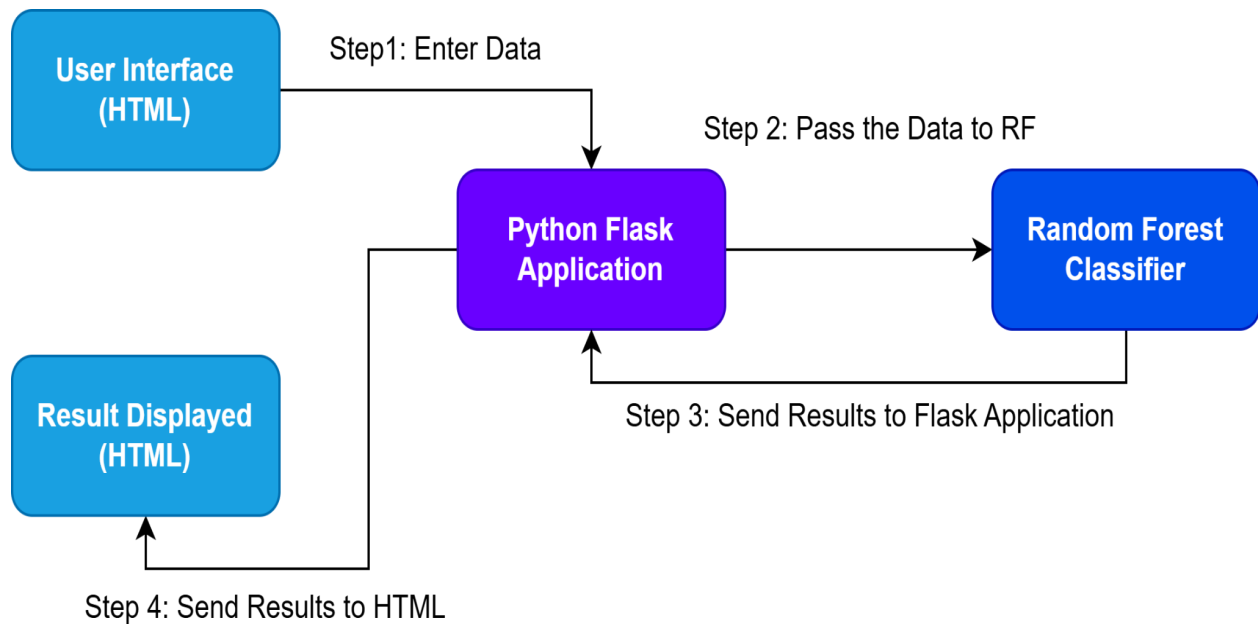


Figure 1.1: Application Workflow

We will focus on both: building a machine learning model for a fake instagram account detector, then creating an API for the model, using Flask, the Python micro-framework for building web applications. This API allows us to utilize predictive capabilities through HTTP requests.

2. Data Information

The dataset used in this project is a curated collection of Instagram account data labeled as either Fake (1) or Genuine (0). It consists of 696 records, each representing an Instagram profile with 11 numerical features extracted from account metadata. This dataset is commonly used for binary classification tasks to train models capable of detecting potentially fake accounts based on their profile characteristics. The table below describes the dataset.

Table 2.1: Dataset Information

Description	Value
Total Records	696
Total Fake Account Records	383
Total Genuine Account Records	313
Total features	11
Target Variable	fake (1 = Fake, 0 = Genuine)

2.1 Attribute Information

The collection is composed of two CSV file, where each line has the following attributes:

Table 2.2: Attribute Information

Feature Name	Description
profile pic	Binary: 1 if profile picture exists, 0 otherwise
nums/length username	Ratio of digits to the total username length
fullname words	Number of words in the fullname
nums/length fullname	Ratio of digits to the total fullname length
name==username	Binary: 1 if name and username are the same, 0 otherwise
description length	Total number of characters in the bio
external url	Binary: 1 if an external link is present, 0 otherwise
private	Binary: 1 if the account is private, 0 if public
#posts	Number of posts shared on the account
#followers	Number of followers
#follows	Number of accounts the user follows

3. Build Machine Learning Model

3.1.1 Import the Dataset

In this part, we import the datasets and merge them which contains the information of 693 instagram account features.

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

1. Load the Dataset

[ ] # Load train and test datasets
import pandas as pd

train = pd.read_csv('/content/drive/MyDrive/Flask deployment/train.csv')
test = pd.read_csv('/content/drive/MyDrive/Flask deployment/test.csv')

[ ] # Merge datasets
df = pd.concat([train, test], ignore_index=True)

#Preview
print("Dataset shape:", df.shape)
df.head()
```

3.1.1 Data Preprocessing

The dataset used here is split into 80% for the training set and the remaining 20% for the test set.

```
2. Preprocess the Dataset

[ ] # Check missing values and data types
df.info()
df.isnull().sum()

# Distribution of target
import seaborn as sns
import matplotlib.pyplot as plt

sns.countplot(x='fake', data=df)
plt.title("Fake vs Real Account Distribution")
plt.show()

[ ] # Define features and target
X = df.drop('fake', axis=1)
y = df['fake']

# Split the data into training and testing
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

3.1.2 Build and Save Model

After data preprocessing, we implement a machine learning model to classify the fake instagram accounts. For this purpose, we implement Random Forest (RF) using scikit-learn. After importing and initializing the RF model we fit into the training dataset and then saved our model using pickle.

```
3. Build and Save the Model

# Import necessary libraries
import pickle
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Predictions and evaluation
rf_preds = rf.predict(X_test)

print("Random Forest")
print(classification_report(y_test, rf_preds))

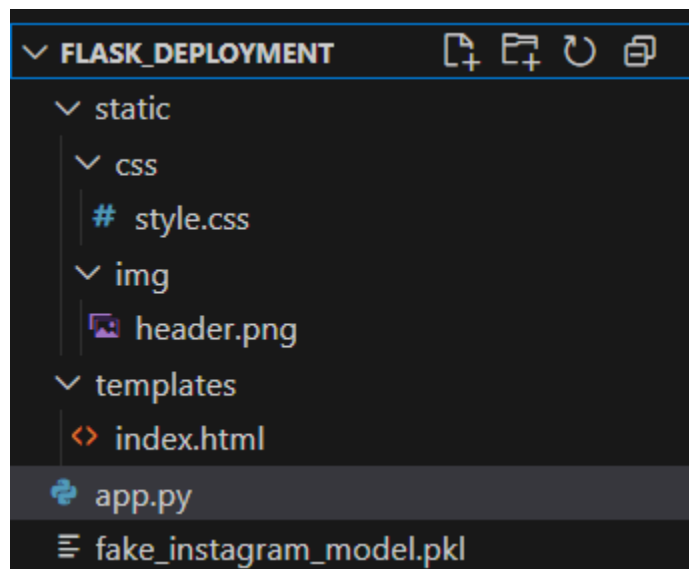
# Save the trained model using pickle
with open('fake_instagram_model.pkl', 'wb') as file:
    pickle.dump(rf, file)

[ ] from google.colab import files
    files.download('fake_instagram_model.pkl')
```

4. Turning Model into Web Application

We develop a web application that consists of a simple web page with a form field that lets us enter account features. After submitting the features to the web application, it will give us a result of fake or genuine.

First, we create a folder for this project called `FLASK_DEPLOYMENT`, this is the directory tree inside the folder. We will explain each file.



The sub-directory templates are the directory in which Flask will look for static HTML files for rendering in the web browser, in our case, we have one HTML file: *index.html*.

4.1 App.py

The *app.py* file contains the main code that will be executed by the Python interpreter to run the Flask web application, it includes the ML code for classifying fake instagram accounts.

```

app.py > ...
You, 54 seconds ago | 1 author (You)
1  from flask import Flask, render_template, request
2  import pickle
3  import numpy as np
4
5  # Load the trained model
6  with open('fake_instagram_model.pkl', 'rb') as f:
7      rf = pickle.load(f)
8
9  app = Flask(__name__)
10
11 @app.route('/')
12 def home():
13     return render_template('index.html')
14
15 @app.route('/predict', methods=['POST'])
16 def predict():
17     # Try to grab the input features and convert to float
18     try:
19         features = [float(x) for x in request.form.values()]
20     except ValueError:
21         # If there is an error in conversion, return the same template with an error message
22         return render_template('index.html', prediction_text="Please enter valid numeric values for all features.")
23
24     # Check if the correct number of features is entered (11 features)
25     if len(features) != 11:
26         return render_template('index.html', prediction_text="Please make sure to enter all 11 features.")
27
28     # Convert features to numpy array for prediction
29     final_features = np.array(features).reshape(1, -1)
30
31     # Model prediction (0 or 1)
32     raw_pred = rf.predict(final_features)[0]
33
34     # Map to friendly text
35     if raw_pred == 0:
36         message = "The account is Genuine ✅"
37     else:
38         message = "The account is Fake ❌"
39
40     # Send the result back to the page
41     return render_template('index.html', prediction_text=message)
42
43 if __name__ == '__main__':
44     app.run(debug=True)
45

```

Figure 4.1: App.py

- We ran our application as a single module; thus we initialized a new Flask instance with the argument `_name_` to let Flask know that it can find the HTML template folder (*templates*) in the same directory where it is located.
- Next, we used the route decorator (`@app.route('/')`) to specify the URL that should trigger the execution of the home function.
- Our *home* function simply rendered the *index.html* HTML file, which is located in the *templates* folder.
- Inside the *predict* function, we access the fake instagram data set, pre-process the text, and make predictions, then store the model. We access the features entered by the user and use our model to make a prediction for its label.
- Lastly, we used the *run* function to only run the application on the server when this script is directly executed by the Python interpreter, which we ensured using the *if* statement with `name_ == '_main'`

4.2 Index.html

The following are the contents of the *index.html* file that will render a form where a user can enter the account features.

```

templates > index.html > ...
You, yesterday | 1 author (You)
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Fake Instagram Predict
6   <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
7 </head>
8 <body>
9
10 <header>
11   
12 </header>
13 <main>
14   <h2>Predict Fake Instagram Account</h2>
15
16   <form action="/predict" method="post">
17     <p>Enter account features:</p>
18
19     <!-- 11 inputs with frontend validation -->
20     <input type="text" name="feature1" placeholder="Have Profile Picture ? (1=yes,0=no)" required pattern="\d+(\.|\d+)?$">
21     <input type="text" name="feature2" placeholder="Ratio of digits to username length" required pattern="\d+(\.|\d+)?$">
22     <input type="text" name="feature3" placeholder="Number of words in fullname" required pattern="\d+(\.|\d+)?$">
23     <input type="text" name="feature4" placeholder="Ratio of digits to fullname length" required pattern="\d+(\.|\d+)?$">
24     <input type="text" name="feature5" placeholder="Is Name = Username ? (1=yes,0=no)" required pattern="\d+(\.|\d+)?$">
25     <input type="text" name="feature6" placeholder="Bio Length" required pattern="\d+(\.|\d+)?$">
26     <input type="text" name="feature7" placeholder="Is URL Present ? (1=yes,0=no)" required pattern="\d+(\.|\d+)?$">
27     <input type="text" name="feature8" placeholder="Is Account Private ? (1=yes,0=no)" required pattern="\d+(\.|\d+)?$">
28     <input type="text" name="feature9" placeholder="Number of Posts" required pattern="\d+(\.|\d+)?$">
29     <input type="text" name="feature10" placeholder="Number of Followers" required pattern="\d+(\.|\d+)?$">
30     <input type="text" name="feature11" placeholder="Number of Follows" required pattern="\d+(\.|\d+)?$">
31
32     <button type="submit" class="predict">Predict</button>
33
34     {% if prediction_text %}
35     <div class="result" {{ 'genuine' if 'Genuine' in prediction_text else 'fake' }}>
36       {{ prediction_text }}
37     </div>
38     {% endif %}
39   </form>
40 </main>
41
42 <footer>
43   © MEHEDI HASAN SHAKIL 2025
44 </footer>
45

```

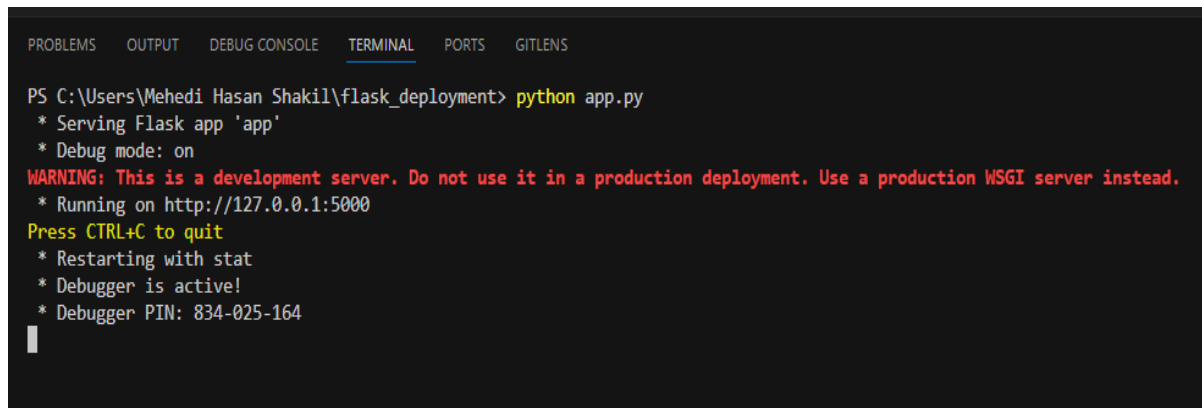
Figure 4.2: index.html

4.3 Style.css

In the header section of *index.html*, we loaded *styles.css* file. CSS is to determine how the look and feel of HTML documents. *styles.css* has to be saved in a subdirectory called *static*, which is the default directory where Flask looks for static files such as CSS.

4.4 Running Procedure

Once we have done all of the above, we can start running the API by either double click *app.py*, or executing the command from the Terminal:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\Mehedi Hasan Shakil\flask_deployment> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 834-025-164

```

Figure 4.3: Command Execution

Now we could open a web browser and navigate to <http://127.0.0.1:5000/>, we should see a simple website with the content like so



Predict Fake Instagram Account

Enter account features:

Have Profile Picture ? (1=yes,0=no)	Ratio of digits to username length
Number of words in fullname	Ratio of digits to fullname length
Is Name = Username ? (1=yes,0=no)	Bio Length
Is URL Present ? (1=yes,0=no)	Is Account Private ? (1=yes,0=no)
Number of Posts	Number of Followers
Number of Follows	

Predict

© MEHEDI HASAN SHAKIL 2025

Figure 4.4: Fake Instagram Account Detection Website Page

Now we enter input in the comments form

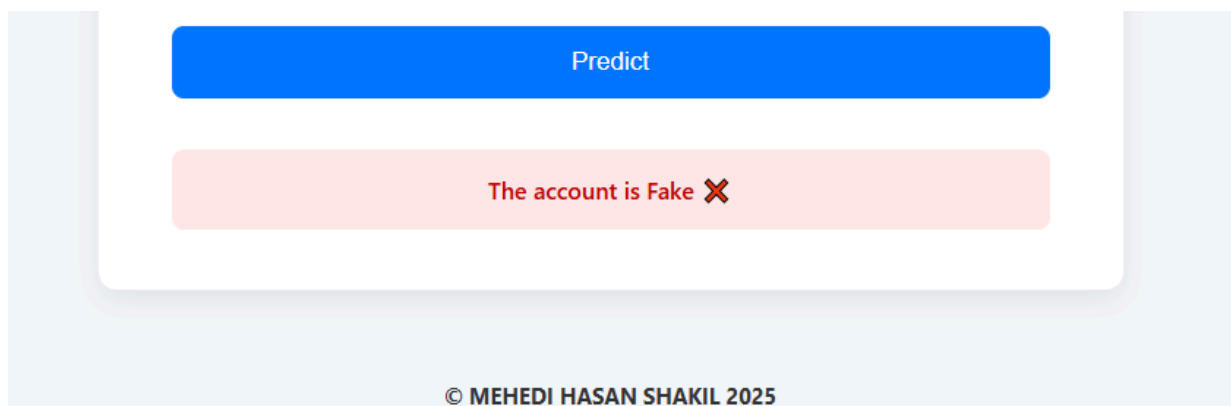


The screenshot shows the 'Fake Instagram Account Detector' web application. At the top is a header with an Instagram logo and the title. Below it is a form titled 'Predict Fake Instagram Account'. The form asks to 'Enter account features:' and contains ten input fields arranged in two columns. The values entered are: 0, 0.27, 2, 0, 0, 0, 0, 0, 1, 15, and 25. A blue 'Predict' button is at the bottom of the form. The footer of the page reads '© MEHEDI HASAN SHAKIL 2025'.

Input Field	Value
1	0
2	0.27
3	2
4	0
5	0
6	0
7	0
8	1
9	15
10	25

Figure 4.5: Input In The Comments Form

After entering the input click the predict button now, we can see the result of our input.



The screenshot shows the result of the prediction. A blue 'Predict' button is at the top. Below it, a red message box displays 'The account is Fake' followed by a red 'X' icon. The footer of the page reads '© MEHEDI HASAN SHAKIL 2025'.

The account is Fake ❌

Figure 4.6: Result of Given Input

5. Model deployment using Heroku

We're ready to start our Heroku deployment now that our model has been trained, the machine learning pipeline has been set up, and the application has been tested locally. There are a few ways to upload the application source code on Heroku. The easiest way is to link a GitHub repository to your Heroku account.

Requirement.txt

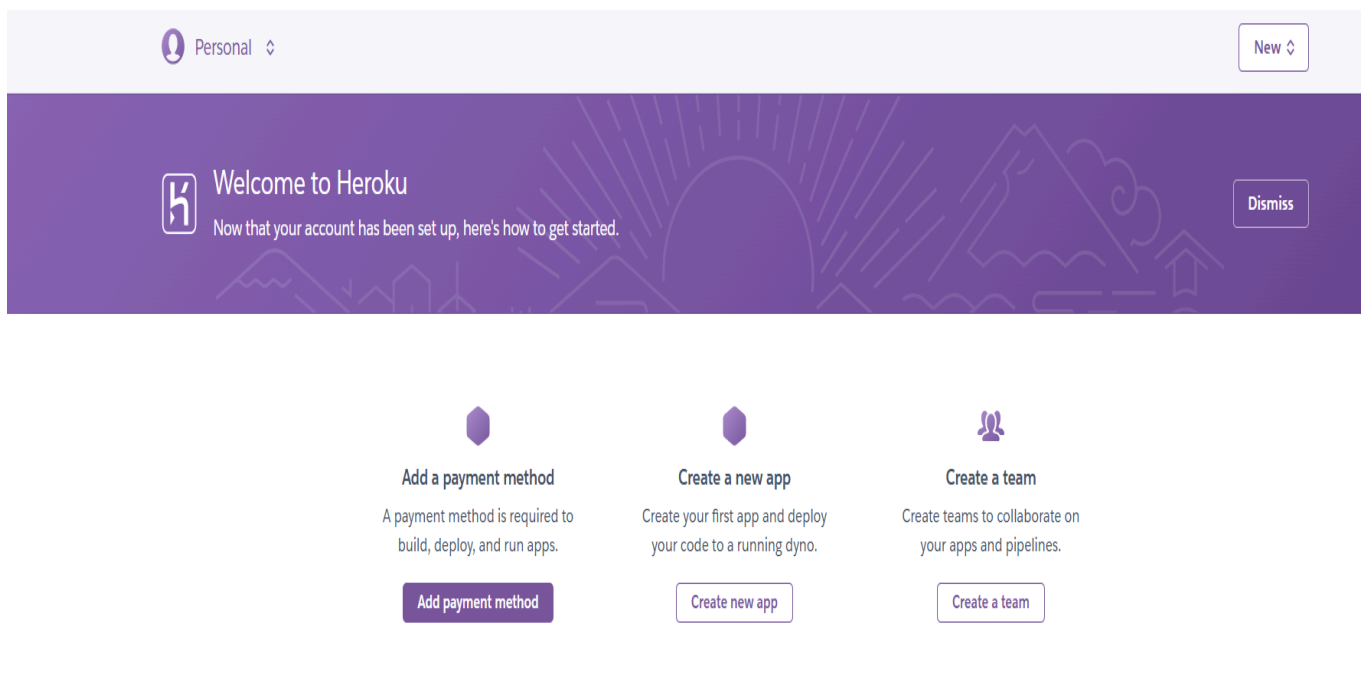
It is a text file containing the python packages required to execute the application.

5.1 Steps for Model Deployment Using Heroku

Once we uploaded files to the GitHub repository, we are now ready to start deployment on Heroku.

Follow the steps below:

1. After signing up on **heroku.com** then click on **Create new app**.



2. Enter App name and region

Create New App

App name

Give this app a globally unique name. For example, acme-production-app.

fake-instagram-detector-api

✓

fake-instagram-detector-api is available

Location

Choose a Common Runtime region for this app. [Learn more.](#)

Common Runtime
CEDAR

United States

Common Runtime
CEDAR

Europe

☒

Add this app to a pipeline

Pipelines form a continuous deployment workflow and enable additional features. All apps in the same pipeline must be the same generation. [Learn more.](#)

Choose a pipeline

⌵

Create a new application named **fake-instagram-detector-api** as a personal app.

Cancel

Create app

3. Connect to the GitHub repository where code is I uploaded.

Deployment method

Heroku Git

Use Heroku CLI

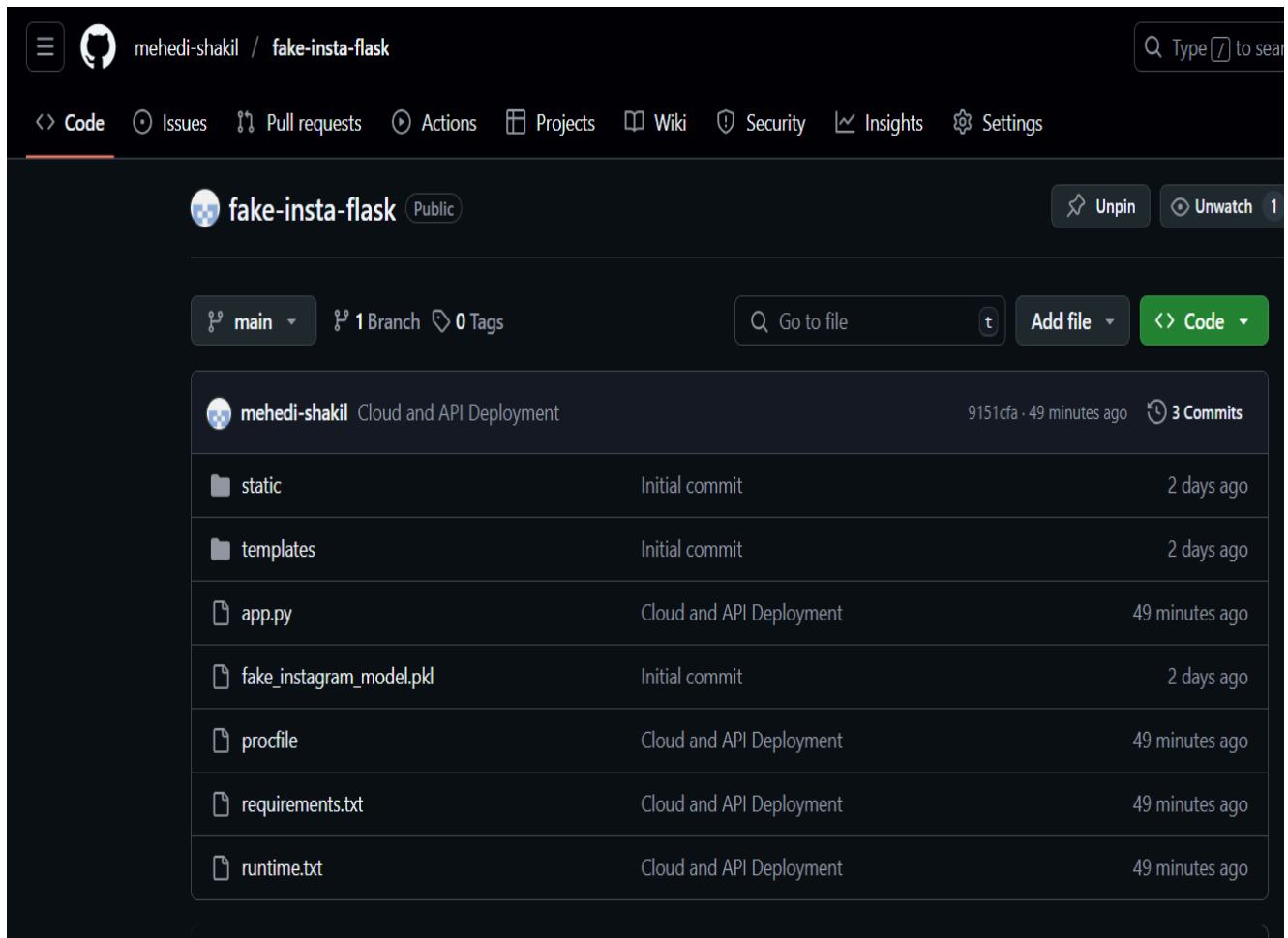
GitHub

Connected ✓

Container Registry

Use Heroku CLI

After that I choose the repository where I upload the code.



4. Deploy branch

Manual deploy

Deploy the current state of a branch to this app.

Deploy a GitHub branch

This will deploy the current state of the branch you specify below. [Learn more.](#)

Choose a branch to deploy

main

Deploy Branch

5. After waiting 1 to 2 minutes our application is ready

Manual deploy

Deploy the current state of a branch to this app.

Deploy a GitHub branch

This will deploy the current state of the branch you specify below. [Learn more](#).

Choose a branch to deploy

 main

Deploy Branch

Receive code from GitHub



Build main b3c3ae21



Release phase



Deploy to Heroku



Your app was successfully deployed.

 View

The app is published at

<https://fake-instagram-detector-api-22db3f7361dd.herokuapp.com/>